Language Rules and Grammar Used in the Compiler

Overview

The compiler uses a custom-designed subset language based on React Function Components and JSX syntax.
The language rules were intentionally simplified to focus on demonstrating compiler construction concepts while still solving a real-world framework translation problem.

Language Rules

1. Component Declaration Rule

A component must be declared using the function keyword.

Syntax

```
function ComponentName() {
return <h1>Hello</h1>
}
```

Rules

- Component names must start with a letter.
- Only one component is allowed per file.
- The component must contain a return statement.

2. JSX Return Rule

The return statement must return valid JSX.

Supported JSX

```
<h1>Hello</h1>
```

```
<div>
<h1>Hello</h1>
</div>
```

Rules

- Opening and closing tags must match.
- Nested tags are allowed.
- Text nodes are supported.
- Self-closing tags are currently not supported.

3. Identifier Rules

Identifiers represent component names.

Rules

- Must start with a letter or underscore.
- Can contain letters, numbers, and underscores.
- Cannot use reserved keywords.

Valid Examples

App

Header
User_Profile

Invalid Examples

123App
function
return

4. Reserved Keywords

The compiler defines reserved keywords:

function
return

These keywords cannot be used as identifiers.

5. JSX Grammar Rules

The JSX grammar used by the compiler follows:

JSX_ELEMENT
→ OPEN_TAG CONTENT CLOSE_TAG

Example

<h1>Hello</h1>

6. Nested JSX Rule

Nested elements are supported.

Example

<h1>Hello</h1>

Grammar

ELEMENT
→ JSX_ELEMENT
| JSX_ELEMENT ELEMENT

7. Semantic Rules

The semantic analyzer validates:

- Component existence
- Valid JSX structure
- Matching opening and closing tags
- Presence of return statement
- Correct component naming

Example Semantic Error

```
function App() {
return <h1>Hello</div>
}
```

Output

Semantic Error:
Mismatched JSX tags

8. Translation Rules

The compiler converts React structures into Angular structures.

React Input

```
function App() {
return <h1>Hello</h1>
}
```

Angular Output

```
@Component({
selector: 'app-app',
template: `<h1>Hello</h1>`
})
export class AppComponent {}
```

Grammar Definition (Simplified BNF)

PROGRAM → FUNCTION_DECLARATION

FUNCTION_DECLARATION
→ "function" IDENTIFIER "(" ")" "{"
RETURN_STATEMENT
"}"

RETURN_STATEMENT
→ "return" JSX_ELEMENT

JSX_ELEMENT → OPEN_TAG CONTENT CLOSE_TAG

OPEN_TAG → "<" IDENTIFIER ">"

CLOSE_TAG → "</" IDENTIFIER ">"

CONTENT → TEXT
| JSX_ELEMENT
| TEXT JSX_ELEMENT

Why These Rules Were Chosen

The rules were intentionally designed to:

- Keep parsing manageable
- Demonstrate real compiler concepts
- Support JSX transformation
- Simulate real framework migration
- Maintain project originality

The subset approach also makes the compiler easier to test, debug, and extend in future versions.